DESENVOLVIMENTO BACK-END

# Projeto e Construção de APIs RESTful

## Unidade III · Um contrato claro entre o back-end e todos os seus clientes

REST · URIs · Richardson · Verbos · Status · RFC 9457 · Validação · DTO · OpenAPI

# 🎯 Objetivos de aprendizagem

🌐

### Compreender

recursos, representações e as restrições do REST

🧭

### Aplicar

boas práticas de URIs, verbos, parâmetros e versionamento

📶

### Analisar

uma API pelo modelo de maturidade de Richardson

🚦

### Diferenciar

códigos de status como 401 × 403 e 422 × 409

🧯

### Explicar

o tratamento centralizado de erros (RFC 9457) e o correlation ID

📘

### Relacionar

validação, DTO, paginação e documentação OpenAPI

# 📲 Um back-end, muitos clientes

O mesmo sistema acadêmico atende:

- o **portal web** e o **app do aluno**
- o **sistema financeiro**
- a plataforma de uma **empresa parceira**

Cada um escrito em uma **linguagem diferente**.
Como todos conversam de forma **previsível**?

📋

**Analogia: o cardápio**

No restaurante, o **cardápio** é o contrato entre salão e cozinha: diz o que pode ser pedido, como pedir e o que vem no prato. A **API** é o cardápio do back-end.

Uma **API** é um **contrato**: boas APIs seguem convenções conhecidas, comunicam erros de forma padronizada e são **bem documentadas**.

# 🌐 REST: recursos e representações

📦

### Recurso

qualquer informação que possa ser **nomeada**: aluno, curso, matrícula, "matrículas ativas do semestre"

🔗

### URI

o **identificador** do recurso: `/api/v1/alunos/42`

📄

### Representação

o **documento** (geralmente JSON) que descreve o estado atual do recurso




💡 REST foi descrito por **Roy Fielding** em sua tese de 2000. O cliente **nunca manipula o recurso** diretamente: ele troca **representações** desse recurso.

# 🧱 As restrições do REST

### 🔀 Cliente-servidor

interface e armazenamento evoluem de forma **independente**

### 🧠 Sem estado

cada requisição traz **tudo** o que é preciso para ser entendida

### 🗃️ Cache

as respostas dizem se podem ser **reaproveitadas**

### 🎛️ Interface uniforme

URIs + **métodos HTTP** padronizados

### 🥞 Sistema em camadas

o cliente não sabe se fala com o servidor ou com um **proxy**

### 📜 Código sob demanda

restrição **opcional**: o servidor envia código ao cliente

💡 **Stateless** não quer dizer "sem dados": o servidor não guarda o **contexto da conversa**. Para provar quem é, o cliente envia um **token** em **toda** requisição.

# 🧭 Boas práticas de URIs e versionamento

| ❌ Inadequado | ✅ Adequado | Motivo |
|---|---|---|
| `GET /getAlunos` | `GET /api/v1/alunos` | o **verbo HTTP** já indica a ação |
| `POST /alunos/excluir/42` | `DELETE /api/v1/alunos/42` | use o **método correto** |
| `GET /aluno/42` | `GET /api/v1/alunos/42` | coleções no **plural** |
| `GET /alunosDoCurso3` | `GET /api/v1/alunos?curso=3` | filtros na **query string** |

💡 Ação que não é CRUD? Modele como **sub-recurso**: `POST /matriculas/7/cancelamento`. Mudança **incompatível**? Crie a **v2** e avise com os cabeçalhos `Deprecation` e `Sunset`.

# 📶 O modelo de maturidade de Richardson

**Nível 0**
pântano do POX
→
**Nível 1**
recursos
→
**Nível 2**
verbos + status
→
**Nível 3**
hipermídia

- **0:** um único endpoint para tudo: `POST /api` com `{"acao": "listarAlunos"}`
- **1:** cada recurso tem sua URI, mas tudo é `POST`: `POST /api/alunos/listar`
- **2:** `GET /api/alunos` → `200`; `POST` → `201`: **o padrão do mercado**
- **3 · HATEOAS:** a resposta traz **links** das próximas ações: "cancelar", "disciplina"

A API acadêmica está no **nível 2**: atingi-lo de forma **consistente** já é sinal de uma API de **boa qualidade**.

# 🗣️ Verbos e parâmetros

| Método | Uso | Idempotente | Sucesso |
|---|---|---|---|
| `GET` | ler | sim | `200` |
| `POST` | criar | **não** | `201` |
| `PUT` | substituir tudo | sim | `200` / `204` |
| `PATCH` | alterar parte | não* | `200` |
| `DELETE` | remover | sim | `204` |

📍

### Onde vai cada dado?

**Rota:** identifica o recurso · `/alunos/42`
**Query:** filtra, ordena, pagina · `?curso=3&page=2`
**Corpo:** dados para criar ou alterar
**Cabeçalho:** metadados e credenciais · `Authorization`

⚠️ **Nunca** use `GET` para alterar dados ( `GET /alunos/42/excluir` ): robôs e proxies repetem GETs, e um simples link visitado poderia **apagar registros**. Dados sensíveis **nunca** vão na URL.

# 🚦 Códigos de status no contrato da API

| Código | Quando usar |
|---|---|
| `200` OK | sucesso com corpo |
| `201` Created | criou; envie `Location` |
| `204` No Content | sucesso sem corpo |
| `404` Not Found | recurso inexistente |
| `422` Unprocessable | dados **inválidos** |

| Código | Quando usar |
|---|---|
| `401` Unauthorized | **não sei quem você é** |
| `403` Forbidden | **sei, mas você não pode** |
| `409` Conflict | sem vagas, duplicidade |
| `429` Too Many Requests | limite excedido |
| `503` Unavailable | banco fora do ar |

Devolver **200 OK com uma mensagem de erro** no corpo é uma das **piores práticas** em APIs: o cliente decide o que fazer **pelo código**.

# 🧯 Tratamento centralizado de erros

**Service**
lança exceção

→

**VagasEsgotadas-Exception**
regra de negócio

→

**Tratador global**
bootstrap/app.php

→

**409 · problem+json**

❌

### Sem centralizar

`try/catch` em **todo** controller, cada erro com um **formato diferente**

✅

### Com tratador global

exceções de **domínio** que não sabem nada de HTTP, convertidas em **um só lugar**

💡 O mesmo padrão vale para os erros do framework: **404**, **405**, **401** e **403** também saem no formato da RFC 9457, só nas rotas `api/*`.

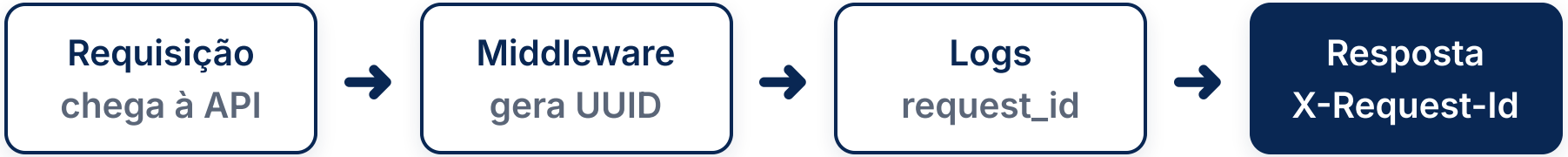
# 📄 O erro padronizado: RFC 9457

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json          # tipo de mídia dos "problem details"
X-Request-Id: 7f3c2a90-...                       # identificador de correlação

{
  "type": "https://api.faculdade.edu.br/erros/vagas_esgotadas",  # URI que identifica o tipo de erro
  "title": "Regra de negócio violada",           # resumo legível e estável
  "status": 409,                                 # repete o código HTTP
  "detail": "Não há vagas disponíveis na disciplina Inteligência Artificial.",  # o caso concreto
  "codigo": "VAGAS_ESGOTADAS"                    # extensão: código curto para o app decidir
}
```

📌 **O que demonstra:** os campos `type`, `title`, `status` e `detail` seguem a **RFC 9457**, e o app decide pelo `codigo`, não pelo texto. 🔒 Em erros **500**, devolva só uma mensagem **genérica** + o ID; pilha e SQL vão para o **log** (`APP_DEBUG=false`).

# 🔎 Correlation ID: rastreando o "deu erro"




- Cada requisição recebe um **identificador único**
- Ele entra em **todas as linhas de log** daquela requisição
- Volta ao cliente no cabeçalho `X-Request-Id` e pode aparecer na **mensagem de erro**
- Entre vários serviços, o mesmo ID é **repassado** em todas as chamadas

O usuário informa o ID e o suporte encontra **na hora** o que aconteceu: é a base da **observabilidade**.

# 🔄 O caminho dos dados: validação, DTO e Resource

**Form Request**
sanitiza + valida
→
**DTO**
dados tipados
→
**Service**
regras
→
**Resource**
JSON público

| | Validação de entrada | Regra de negócio |
|---|---|---|
| **Pergunta** | os dados estão **bem formados**? | a operação é **permitida agora**? |
| **Onde fica** | Form Request | Service |
| **Falha** | `422` · nota fora de 0 a 10 | `409` · sem vagas na disciplina |

💡 **Valide sempre no back-end**: a validação do front-end melhora a experiência, mas é contornada com um simples `curl`.

# 🔧 O controller depois da Unidade III

```php
// app/Http/Controllers/Api/V1/AlunoController.php
public function store(StoreAlunoRequest $request): JsonResponse  // 422 automático se inválido
{
    $dto = CadastrarAlunoDTO::fromArray($request->validated());  // só dados validados, tipados
    $aluno = $this->cadastro->cadastrar($dto);                   // regra de negócio no Service

    return (new AlunoResource($aluno))                           // representação pública
        ->response()
        ->setStatusCode(201)                                     // 201 Created
        ->header('Location', route('alunos.show', $aluno));      // onde o recurso novo vive
}

public function show(Aluno $aluno): AlunoResource                // route model binding: 404 automático
{
    return new AlunoResource($aluno->load('curso'));             // senha e campos internos ficam de fora
}
```

📌 **O que demonstra:** comparado ao controller da Unidade II, a validação foi para o **Form Request**, os dados viajam no **DTO**, a resposta é montada pelo **Resource** e o **route model binding** já devolve 404. Cada linha tem **uma responsabilidade**.

# 📚 Paginação e documentação com OpenAPI

📑

### Paginação padronizada

Toda listagem devolve `data`, `links` (first, next...) e `meta` (página, total).

- limite de **100 por página**
- ordenação só por campos de uma **lista fechada**
- tabelas enormes: `cursorPaginate()`

📘

### OpenAPI / Swagger

Descreve endpoints, parâmetros, **schemas** e status em YAML ou JSON.

- **Swagger UI**: "Try it out" no navegador
- **contract-first**: contrato antes do código
- **code-first**: gerado do código (Scramble)

Documentar é **parte do desenvolvimento**: o `openapi.yaml` fica no repositório, é revisado junto com o código e **validado no CI**.

# 🍽️ APIs no dia a dia

| Conceito | No dia a dia | Na API acadêmica |
|---|---|---|
| **API / contrato** | o **cardápio** do restaurante | `openapi.yaml` |
| **Recurso × representação** | a pessoa × a **foto 3×4** dela | aluno 42 × JSON do aluno 42 |
| **Stateless** | o caixa que pede o **cartão** a cada compra | token em **toda** requisição |
| **401 × 403** | sem crachá × crachá sem acesso à sala | sem token × aluno lançando nota |
| **Correlation ID** | o **número do protocolo** do atendimento | `X-Request-Id` |

💡 Use a analogia para **entender** e o termo técnico para **explicar**: é ele que aparece na documentação e nas entrevistas.

# 🌍 Aplicação em sistemas reais

📱

### App do aluno

front-end e back-end em **paralelo**, a partir de um contrato OpenAPI e de um **servidor mock**

💳

### Pagamentos

o provedor avisa por **webhook**; repetir um `POST` pode cobrar **duas vezes**

📮

### Consulta de CEP

o back-end também é cliente: **timeout**, **retry** e resposta traduzida

**Autenticado?** senão 401 → **Permitido?** senão 403 → **Dados e regra ok?** senão 422 / 409 → **200 / 201**

💡 Esse roteiro ajuda a escolher o **código de status** certo para qualquer endpoint da API acadêmica.

# Síntese dos conceitos

1. **REST**: recursos, URIs e **representações**
2. **Stateless**: cada requisição traz tudo o que precisa
3. **URIs** no plural, sem verbos, com **/v1**
4. **Richardson nível 2**: verbos e status corretos

1. **401 × 403**, **422 × 409**: o código é contrato
2. **RFC 9457** + tratador global + **correlation ID**
3. **Form Request ➜ DTO ➜ Resource**
4. **Paginação** e **OpenAPI** como parte do código

Uma boa API é **previsível**: quem a consome não precisa **adivinhar** nada.

# 📝 Reflexão

## 🧩 Desafio: complete o contrato

```
______ /api/v1/matriculas/7/cancelamento   # qual verbo?   → status?
GET    /api/v1/alunos?_______________      # curso 3, página 2
POST   /api/v1/notas  {"nota": 11}         # nota inválida → status?
POST   /api/v1/matriculas                  # disciplina sem vagas → status?
GET    /api/v1/alunos/9999                 # aluno inexistente → status?
```

## 💬 Reflita

1. Por que repetir um `DELETE` é seguro, mas repetir um `POST` não é?
2. Qual a diferença, para o usuário, entre um **401** e um **403**?
3. Que informações **nunca** devem aparecer em uma resposta de erro **500**?